

Containers and Kubernetes TASK – 2

- 1. Install Minikube on your local machine and start a new Kubernetes cluster using the minikube start command. Take a screenshot of your running cluster status.**

Ans :

- Make sure Docker Desktop is installed and running.
- Open your browser and download Minikube from the official Minikube website.
- Install Minikube by following the installation instructions for your operating system.
- Open Command Prompt or Terminal.
- Check that Minikube is installed:
 - minikube version
 - You should see something similar to:
 - minikube version: v1.xx.x
- Start the Kubernetes Cluster
 - Start a new Kubernetes cluster:
 - minikube start
 - Wait for Minikube to finish. You should see messages indicating that the control plane and Kubernetes components have started.
- Check the cluster status:
 - minikube status
 - You should see something similar to:
 - type: Control Plane

- host: Running
- kubelet: Running
- apiserver: Running
- kubeconfig: Configured

- Take a screenshot of this terminal output.

This is the screenshot you can submit for the assignment.

- Commands to run
- Run these one by one:
- minikube version
- minikube start
- minikube status

2. Write a YAML configuration file named music-pod.yaml to deploy a pod running the nginx image with the pod name set to spotify-nginx. Apply this configuration using kubectl.

Ans :

- Create the YAML file
- Open Notepad (Windows) or any text editor.
- Create a new file named:
- music-pod.yaml
- Put this content inside the file:
- apiVersion: v1
- kind: Pod
- metadata:
- name: spotify-nginx
- spec:
- containers:
 - name: nginx
- image: nginx
- Save the file.
- Open Command Prompt / Terminal

- Go to the folder where you saved music-pod.yaml.
- For example, if you saved it on your Desktop:
- cd Desktop
- Apply the YAML configuration
- Run:
- kubectl apply -f music-pod.yaml
- You should see:
- pod/spotify-nginx created
- Check the Pod
- Run:
- kubectl get pods
- You should see something similar to:
- NAME READY STATUS RESTARTS AGE
- spotify-nginx 1/1 Running 0 10s
- Check the Pod details
- You can also run:
- kubectl describe pod spotify-nginx
- Commands you need
- kubectl apply -f music-pod.yaml
- Then:
- kubectl get pods

**3. Use kubectl to check the status of your spotify-nginx pod and display its details. Then, delete the pod using the appropriate kubectl command.

Hint: Use kubectl get pods and kubectl describe pod, then kubectl delete pod.**

Ans :

- Check the Pod Status
- Open Command Prompt / Terminal and run:
- kubectl get pods

- You should see:
- NAME READY STATUS RESTARTS AGE
- spotify-nginx 1/1 Running 0 1m
- The important part is:
- spotify-nginx 1/1 Running
- ---
- ○ Display Pod Details
- Run:
- kubectl describe pod spotify-nginx
- This will show details such as:
- Pod name
- Namespace
- Status
- IP address
- Container name
- Nginx image
- Events
- You don't need to type anything else—just read the information displayed.
- ---
- Delete the Pod
- Run:
- kubectl delete pod spotify-nginx
- You should see:
- pod "spotify-nginx" deleted
- ---
- Confirm the Pod Was Deleted
- Run:

- kubectl get pods
 - You should no longer see spotify-nginx in the list.
 - Commands in order
 - kubectl get pods
 - kubectl describe pod spotify-nginx
 - kubectl delete pod spotify-nginx
 - kubectl get pods
- 4. Create a YAML file named insta-service.yaml to expose your spotify-nginx pod using a Kubernetes Service of type NodePort, making it accessible externally. Apply the service and note the port assigned.

Hint: Use kind: Service and type: NodePort in your YAML.**

Ans :

- Create the YAML file
- Open Notepad or any text editor.
- Create a file named:
- insta-service.yaml
- Put this inside:
- apiVersion: v1
- kind: Service
- metadata:
- name: insta-service
- spec:
- type: NodePort
- selector:
- app: spotify-nginx
- ports:
 - port: 80
 - targetPort: 80
- apiVersion: v1

- kind: Pod
- metadata:
- name: spotify-nginx
- labels:
- app: spotify-nginx
- spec:
- containers:
 - name: nginx
 - image: nginx
- Since you previously deleted the Pod, recreate it:
- kubectl apply -f music-pod.yaml
 - Apply the Service
- Run:
- kubectl apply -f insta-service.yaml
- You should see:
- service/insta-service created
 - Check the Service
- Run:
- kubectl get services
- You should see something like:
- | NAME | TYPE | CLUSTER-IP | EXTERNAL-IP |
|---------------|----------|------------|-------------|
| insta-service | NodePort | 10.x.x.x | <none> |
- 80:30xxx/TCP
- The number after 80: is the NodePort assigned to your Service.
- For example:
- 80:30080/TCP
- Here, 30080 is the assigned NodePort.
- Get the exact assigned port
- You can also run:
- kubectl get service insta-service

- Access the Service with Minikube
- Because you're using Minikube, the easiest way to open it is:
- minikube service insta-service
- This should open the Nginx page in your browser.
- Commands in order
- kubectl apply -f music-pod.yaml
- kubectl apply -f insta-service.yaml
- kubectl get services
- minikube service insta-service

5. Compare deploying a web app using Kubernetes pods and services versus running it directly on a single VM (like a traditional deployment). List 3 advantages of using Kubernetes for scaling an app like Zomato or Flipkart.

Ans :

Advantage	Kubernetes	Single VM	Example
1. Easy scaling	Kubernetes can run multiple copies (Pods) of an application and increase or decrease them as needed.	A single VM has limited capacity. Scaling usually requires manually adding or upgrading servers.	Zomato can run more Pods during busy meal times when the number of orders increases.
2. High availability	If one Pod fails, Kubernetes can restart it or run another	If the application or VM fails, the service may become	Flipkart can run multiple application Pods so one failed Pod

	Pod to keep the application available.	unavailable until it is fixed.	doesn't necessarily stop the entire service.
3. Automatic management	Kubernetes can automatically manage, restart, and distribute Pods across available resources.	Administrators generally need to manage processes and scaling manually.	During a Flipkart sale, Kubernetes can manage additional application instances as demand increases.